



# Creating a Containerized Microservice

This second episode advances the microservices architecture by integrating Amazon Elastic Container Service (ECS) on top of the previously established VPC foundation. It begins with a brief recap of the VPC setup from earlier stages, then introduces ECS as a container orchestration platform. The focus shifts to creating an ECS cluster and defining a task for a simple client service, emphasizing Fargate for serverless execution. Explanations cover why certain configurations, like CPU and memory settings, are chosen, and build progressively from basic concepts to dynamic, code-based implementations.

## Recapping the VPC Foundation

Before deploying containers, the VPC provides the networking backbone for microservices, isolating resources and controlling traffic. In prior configurations, a VPC was created with public and private subnets spread across availability zones for resilience. Public subnets enable internet access via an internet gateway, while private subnets use a NAT gateway for outbound connections without exposure. Route tables and associations direct traffic, and data sources dynamically fetch availability zones.

To reinstate this setup, execute `terraform apply` in the project directory. This command recreates all VPC resources declaratively, demonstrating Terraform's idempotency—resources are rebuilt exactly as defined without manual intervention. The output lists resources like the VPC, subnets, gateways, and route tables, confirming the network is ready for ECS tasks.

## Why Reapply the VPC?

Reapplying ensures consistency across environments and avoids drift from manual changes. It's efficient because Terraform plans changes first, only creating what's missing. This step verifies the infrastructure-as-code approach, where the VPC file encapsulates all configurations, including counts for multiple subnets and CIDR offsets to prevent overlaps.

## Introduction to Amazon ECS

Amazon Elastic Container Service (ECS) orchestrates Docker containers, managing

deployment, scaling, and operations for microservices. It abstracts container runtime details, allowing focus on application logic. ECS supports two launch types: EC2 (user-managed instances) and Fargate (serverless, AWS-managed infrastructure). Fargate is ideal for beginners, eliminating server provisioning and patching, with billing based on vCPU and memory usage.

## Key ECS Components

- **Cluster:** A logical grouping of tasks or services, serving as the top-level container for resources.
- **Task Definition:** A blueprint for tasks, specifying containers, CPU/memory, networking, and environment variables. It's like a Docker Compose file but ECS-specific.
- **Task:** A running instance of a task definition, executing one or more containers.
- **Service:** Maintains a desired number of tasks, handling scaling, load balancing, and health checks.

ECS integrates with VPCs for secure networking, using AWSVPC mode to assign ENIs (Elastic Network Interfaces) to tasks for direct VPC placement.

## Choosing Fargate Over EC2

Fargate simplifies operations by handling underlying compute, making it suitable for microservices where focus is on code rather than infrastructure. It scales automatically based on task demands but requires precise CPU/memory specifications to control costs. EC2 offers more customization but adds management overhead.

## Creating the ECS Cluster

The cluster is the foundational ECS resource, requiring minimal configuration. Start with a single cluster to host all services, named for easy identification.

## Cluster Configuration Basics

The cluster name ties resources together. No advanced settings like capacity providers are needed initially, as Fargate handles provisioning implicitly.

## Defining the Cluster in Terraform

Create a dedicated file ( `ecs-cluster.tf` ) for organization. The resource references project variables for consistency.

```
# File: ecs-cluster.tf

resource "aws_ecs_cluster" "main" {
  name = var.default_tags.project
}
```

Here, `var.default_tags.project` ensures naming aligns with other resources, promoting traceability. Apply this with `terraform apply` to provision the cluster, visible in the AWS console under ECS > Clusters.

## Why Minimal Configuration?

Clusters in ECS are lightweight; complexity arises in tasks and services. Omitting extras like logging configurations keeps the setup focused, adding them later if needed for monitoring.

## Defining the ECS Task

Tasks define how containers run. For a client microservice, use a task definition with Fargate compatibility, specifying resources and container details.

## Building Up to the Task Definition

Start by understanding requirements: The task runs a single container from a public Docker image (fake-service for simulation). Specify Fargate to avoid server management. Set modest CPU (256 units) and memory (512 MB) to minimize costs while supporting the service—Fargate bills per vCPU-hour and GB-hour, so these limits cap usage.

Network mode "awsvpc" integrates with the VPC, assigning private IPs to tasks. Container definitions, required in JSON, outline the image, ports, and environment.

## Prerequisites for Task Creation

Ensure the VPC is applied, as tasks need subnets and security groups (added later). Test the Docker image locally if possible to verify ports and environment variables.

## Using jsonencode for Container Definitions

Container definitions must be JSON, but writing JSON in HCL is error-prone. Terraform's `jsonencode` function converts HCL structures to JSON, improving readability.

The structure is an array of objects, each a container. For one container:

- **name**: Identifies the container.
- **image**: Docker image URI.
- **cpu**: Reserves CPU (0 allows proportional sharing).
- **essential**: Marks as critical for task health.
- **portMappings**: Maps host to container ports (9090 for fake-service).
- **environment**: Key-value pairs configuring the app (e.g., service name and message).

Build this progressively: Start with basics, then add ports and env vars.

## Complete Task Definition

Organize in `ecs-task-definition.tf`. The family acts as a versioned name.

```
# File: ecs-task-definition.tf

resource "aws_ecs_task_definition" "client" {
  family                = "${var.default_tags.project}-client"
  requires_compatibilities = ["FARGATE"]
  memory                = 512
  cpu                   = 256
  network_mode          = "awsvpc"

  container_definitions = jsonencode([
    {
      name      = "client"
      image     = "nicholasjackson/fake-service:vm-v0.26.2"
      cpu       = 0
      essential = true

      portMappings = [
        {
          containerPort = 9090
          hostPort       = 9090
          protocol       = "tcp"
        }
      ]

      environment = [
        {
          name  = "NAME"
          value = "client"
        },
        {
          name  = "MESSAGE"
          value = "Hello from the client"
        }
      ]
    }
  ])
}
```

Apply to register the task definition. In the console, it appears under ECS > Task Definitions,

ready for services.

## Why These Settings?

- **CPU=0**: Allows flexible sharing; in multi-container tasks, it proportions resources without hard limits.
- **Ports**: Matches fake-service's listener; hostPort=containerPort for Fargate simplicity.
- **Environment Vars**: Customizes fake-service without code changes, simulating real configs.
- **Omissions**: No volumes or secrets yet, as this is a stateless service. Add if persistence is needed.

## Resource Limits and Scaling Considerations

CPU and memory in tasks set hard limits—containers can't exceed them, preventing overages but risking OOM kills if insufficient. These influence costs: Higher reservations increase bills, even if unused. For scaling, limits are enforced per task; auto-scaling occurs at the service level (covered later), adjusting task counts based on metrics like CPU utilization.

In Fargate, minimums are 0.25 vCPU and 0.5 GB, scaling up to 16 vCPU and 120 GB. Choose based on workload testing to balance performance and cost.

## Defining and Configuring AWS ECS Services

### Intro

This section continues the deployment of a microservices architecture by introducing ECS services, which orchestrate the runtime behavior of containerized tasks. Building on the ECS cluster and task definition from previous steps, it explores how services ensure availability and integrate with networking components like load balancers and security groups. The discussion progresses from basic service configuration to incorporating load balancing for external access, security rules for controlled traffic, and outputs for convenience. Explanations highlight why services differ from standalone tasks, the role of target groups in traffic distribution, and the importance of destroying resources to manage costs.

# Understanding ECS Services

An ECS service manages the lifecycle of tasks derived from a task definition, ensuring they remain running and available. Unlike a standalone task, which runs once and terminates (suitable for batch jobs), a service maintains a desired number of task instances continuously. If a task fails—due to crashes, health issues, or scaling events—the service automatically replaces it, promoting high availability and reliability for long-running microservices like APIs or web applications.

## Why Use Services for Microservices?

Microservices require consistent uptime and scalability. Services handle this by monitoring task health and restarting as needed, integrating seamlessly with load balancers for traffic distribution. This abstraction allows developers to focus on application logic rather than manual restarts or monitoring. For the client service example, a service ensures the fake-service container is always accessible, even under load or failures.

## Configuring the ECS Service

Start with a dedicated file ( `ecs-services.tf` ) for clarity. The service references the cluster and task definition ARNs (Amazon Resource Names), specifying Fargate as the launch type for serverless execution. Set a desired count (e.g., 1 for initial testing) to control replicas.

Network configuration places tasks in private subnets for security, disabling public IPs to prevent direct exposure. Security groups (defined later) control access, and load balancer integration routes traffic to the container.

Build this after scaffolding dependencies like load balancers and security groups, as the service references them.

```
# File: ecs-services.tf

resource "aws_ecs_service" "client" {
  name            = "${var.default_tags.project}-client"
  cluster         = aws_ecs_cluster.main.arn
  task_definition = aws_ecs_task_definition.client.arn
  desired_count   = 1
  launch_type     = "FARGATE"

  load_balancer {
    target_group_arn = aws_lb_target_group.client_alb_targets.arn
    container_name   = "client"
    container_port    = 9090
  }

  network_configuration {
    subnets          = aws_subnet.private.*.id
    assign_public_ip  = false
    security_groups   = [aws_security_group.ecs_client_service.id]
  }
}
```

The `load_balancer` block specifies the target group (a pool of tasks), container name from the task definition, and port (9090 for fake-service). This ensures traffic reaches the correct container. Private subnets enhance security, as tasks don't need public IPs—traffic flows through the load balancer.

## Prerequisites and Dependencies

Before finalizing the service, create the load balancer and security groups. Terraform handles implicit dependencies via references (e.g., ARNs), but explicit ordering via `depends_on` can be added if needed. Validate the task definition applies successfully first, ensuring the blueprint is registered.

## Integrating Load Balancing

To expose the client service externally, use an Application Load Balancer (ALB), which



handles HTTP/HTTPS traffic intelligently. The ALB sits in public subnets, distributing requests to ECS tasks via a target group.

## Why an ALB for User-Facing Services?

ALBs provide layer-7 routing, health checks, and SSL termination, making them ideal for microservices. They ensure even traffic distribution and automatic failover if tasks become unhealthy. For the client service, the ALB provides a single DNS endpoint, abstracting multiple task instances.

## Load Balancer Configuration

Organize in `ecs-loadbalancers.tf`. The ALB uses public subnets for internet access and a security group (defined later) for traffic control. Enable dual-stack IP (IPv4/IPv6) for broader compatibility.

```
# File: ecs-loadbalancers.tf

resource "aws_lb" "client_alb" {
  name_prefix      = "cl-" # 6 char length
  load_balancer_type = "application"
  security_groups  = [aws_security_group.client_alb.id]
  subnets         = aws_subnet.public.*.id
  idle_timeout     = 60
  ip_address_type  = "dualstack"

  tags = { "Name" = "${var.default_tags.project}-client-alb" }
}
```

`name_prefix` ensures uniqueness, and `idle_timeout` closes inactive connections to free resources.

## Target Group for Task Registration

The target group acts as a registry for ECS tasks, allowing the ALB to route traffic. Specify IP targeting for Fargate tasks, which receive unique IPs in the VPC.

```
# File: ecs-loadbalancers.tf

resource "aws_lb_target_group" "client_alb_targets" {
  name_prefix      = "cl-"
  port             = 9090
  protocol         = "HTTP"
  vpc_id           = aws_vpc.main.id
  deregistration_delay = 30
  target_type      = "ip"

  health_check {
    enabled          = true
    path            = "/"
    healthy_threshold = 3
    unhealthy_threshold = 3
    timeout         = 30
    interval        = 60
    protocol        = "HTTP"
  }

  tags = { "Name" = "${var.default_tags.project}-client-tg" }
}
```

`deregistration_delay` gives tasks time to drain connections before removal. The health check pings the root path ("/") every 60 seconds, marking targets healthy after 3 successes or unhealthy after 3 failures. This integrates with the ECS service for automatic replacement of failed tasks.

## ALB Listener for Incoming Traffic

Listeners define how the ALB handles requests. For simplicity, start with HTTP on port 80, forwarding to the target group.

```
# File: ecs-loadbalancers.tf

resource "aws_lb_listener" "client_alb_http_80" {
  load_balancer_arn = aws_lb.client_alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.client_alb_targets.arn
  }
}
```

This forwards all traffic to the client target group. Add HTTPS listeners later for production, with ACM certificates.

## Securing Traffic with Security Groups

Security groups act as virtual firewalls, controlling ingress and egress at the resource level. For the ALB and ECS service, define rules to allow necessary traffic while restricting others.

### Why Security Groups?

They enforce least-privilege access: ALBs allow public inbound but only forward to trusted ECS tasks. ECS tasks permit inbound from the ALB only, and outbound for updates. This layered security prevents unauthorized access.

### ALB Security Group

Allow HTTP from anywhere (for public access) and unrestricted outbound.

```
# File: security-groups.tf

resource "aws_security_group" "client_alb" {
  name_prefix = "${var.default_tags.project}-ecs-client-alb"
  description = "security group for client service application load balancer"
  vpc_id      = aws_vpc.main.id
}

resource "aws_security_group_rule" "client_alb_allow_80" {
  security_group_id = aws_security_group.client_alb.id
  type              = "ingress"
  protocol          = "tcp"
  from_port         = 80
  to_port           = 80
  cidr_blocks       = ["0.0.0.0/0"]
  ipv6_cidr_blocks  = [ "::/0" ]
  description       = "Allow HTTP traffic."
}

resource "aws_security_group_rule" "client_alb_allow_outbound" {
  security_group_id = aws_security_group.client_alb.id
  type              = "egress"
  protocol          = "-1"
  from_port         = 0
  to_port           = 0
  cidr_blocks       = ["0.0.0.0/0"]
  ipv6_cidr_blocks  = [ "::/0" ]
  description       = "Allow any outbound traffic."
}
```

## ECS Service Security Group

Allow inbound from the ALB on port 9090, self-referential traffic (for multi-task communication), and outbound anywhere.

```
# File: security-groups.tf

resource "aws_security_group" "ecs_client_service" {
  name_prefix = "${var.default_tags.project}-ecs-client-service"
  description = "ECS Client service security group."
  vpc_id      = aws_vpc.main.id
}

resource "aws_security_group_rule" "ecs_client_service_allow_9090" {
  security_group_id = aws_security_group.ecs_client_service.id
  type              = "ingress"
  protocol          = "tcp"
  from_port         = 9090
  to_port           = 9090
  source_security_group_id = aws_security_group.client_alb.id
  description = "Allow incoming traffic from the client ALB into the service container port"
}

resource "aws_security_group_rule" "ecs_client_service_allow_inbound_self" {
  security_group_id = aws_security_group.ecs_client_service.id
  type              = "ingress"
  protocol          = "-1"
  self              = true
  from_port         = 0
  to_port           = 0
  description       = "Allow traffic from resources with this security group."
}

resource "aws_security_group_rule" "ecs_client_service_allow_outbound" {
  security_group_id = aws_security_group.ecs_client_service.id
  type              = "egress"
  protocol          = "-1"
  from_port         = 0
  to_port           = 0
  cidr_blocks       = ["0.0.0.0/0"]
  ipv6_cidr_blocks  = [ "::/0" ]
  description       = "Allow any outbound traffic."
}
```

`source_security_group_id` restricts inbound to the ALB only, enhancing isolation.

## Outputting Key Values

Outputs expose resource attributes post-apply, like the ALB DNS for easy access without console navigation.

## Why Outputs?

They simplify workflows, outputting values to the terminal for scripting, testing, or quick reference. In modules, outputs pass data between configurations.

```
# File: outputs.tf

output "client_alb_dns" {
  value      = aws_lb.client_alb.dns_name
  description = "DNS name of the AWS ALB for Client service"
}
```

After `terraform apply`, the DNS appears in the output, ready for browser testing.

## Resource Cleanup

To avoid unnecessary costs, destroy resources with `terraform destroy` after testing. This removes all provisioned items, as the configuration remains in code for recreation. Always confirm the plan to prevent accidental deletions in production-like setups.